FREE SAMPLE

AI Prompt Pack for Developers

5 battle-tested prompts from the full 50-prompt pack

Works with ChatGPT · Claude · Gemini · Any LLM

50 Prompts

8 Categories

∞ LLMs

toolsforbuilders.gumroad.com

How to use this pack

Each prompt below is ready to copy and paste into ChatGPT, Claude, or any LLM. Replace the text in brackets [ ] with your specific context. The prompts are structured to produce output that's immediately useful — not a vague starting point.

This is a free sample of the full pack, which includes 50 prompts across 8 categories: Code Review, Documentation, Architecture, Testing, Refactoring, DevOps, Learning, and Client Communication.

CODE REVIEW

PROMPT 01 / 05

Thorough Code Review

A vague 'review my code' prompt produces vague feedback. This prompt forces the model to be specific: identify the exact issue, explain why it's a problem, and propose a concrete fix.

THE PROMPT:

```
Review this [language] code for: bugs and logical errors, edge cases not handled, performance issues, security vulnerabilities, and readability problems.

For each issue found:
1. Quote the specific line(s) with the problem
2. Explain why it's an issue
3. Provide a corrected version with a brief explanation

Code:
[paste your code here]
```

Tip: Add 'Focus only on critical issues' if you want a concise review, or 'Be exhaustive' for a deep audit.

DOCUMENTATION

PROMPT 02 / 05

Professional README Generator

Most AI-generated READMEs are generic. This prompt provides the structure and context the model needs to produce a README that looks professionally written.

THE PROMPT:

```
Write a professional README.md for this project. Include the following sections:

- Project title and one-line description
- Overview (2-3 sentences: what it does, who it's for, why it exists)
- Prerequisites
- Installation (step-by-step commands)
- Usage (with real code examples)
- Configuration options (if any)
- Contributing guide
- License

Project details:
- Name: [project name]
- Tech stack: [languages/frameworks]
- What it does: [description]
- Target user: [developer type or use case]
```

Tip: Include a sample of your actual code and the model will match the style and terminology to your project.

DEBUGGING

PROMPT 03 / 05

Rubber Duck Debugger

Instead of dumping code and waiting for an answer, this prompt turns the model into an active debugging partner that guides you to the solution through questions — which often helps you find the bug yourself.

THE PROMPT:

```
I have a bug I cannot solve. Act as a senior debugging partner and help me work through it systematically.

Ask me questions one at a time to understand:
```

```
1. What the code is supposed to do
2. What it's actually doing
3. What I've already tried
4. The relevant code sections
5. The environment and dependencies

Based on my answers, form and test hypotheses. Don't guess — ask until you have enough information to make a confident diagnosis.

Start by asking me to describe the problem in one sentence.
```

Tip: This works especially well for elusive bugs where you have a hunch but can't pin it down.

REFACTORING

PROMPT 04 / 05

Legacy Code Refactor

Without specific instructions, refactoring prompts produce cosmetic changes. This prompt forces the model to first understand intent, then surface real structural problems before suggesting improvements.

THE PROMPT:

```
Refactor this [language] code. Before proposing changes:

1. Explain in plain English what this code currently does
2. Identify any anti-patterns, code smells, or technical debt
3. List the specific improvements you'll make and why

Then provide the refactored version with inline comments explaining the key changes made.

Priorities: readability > maintainability > performance (unless performance is the stated issue).

Code:
[paste your code here]
```

Tip: Add 'Do not change the external interface or function signatures' to prevent breaking changes.

CLIENT COMMUNICATION

PROMPT 05 / 05

Technical-to-Business Translator

Developers consistently overestimate how much business stakeholders understand. This prompt bridges the gap by producing a clear, jargon-free summary that non-technical people can act on.

THE PROMPT:

```
Translate this technical update into a clear summary for a
non-technical business stakeholder.

Rules:
- No technical jargon or acronyms (if unavoidable, define them in plain
language)
- Focus on business impact, not implementation details
- Use plain language a smart non-developer would understand
- Maximum 150 words
- End with any action required from the stakeholder (if any)

Format: Subject line + body paragraph + action item (if applicable)

Technical update:
[paste your technical update here]
```

Tip: Works equally well for status update emails, incident post-mortems, and sprint review summaries.

Want all 50 prompts?

The full AI Prompt Pack for Developers includes:

★ 50 prompts across 8 categories

- Code Review & Debugging
- Documentation & README
- Architecture & System Design
- Testing & QA
- Refactoring & Code Quality
- Git, DevOps & CI/CD
- Learning & Concept Explanation
- Client & Stakeholder Communication

€9.99 — one-time payment — instant download

toolsforbuilders.gumroad.com/l/usaipu

Also available: Freelancer HQ — Complete Business OS
toolsforbuilders.gumroad.com/l/ouatlw | €14.99